Boletín 1
JavaScript moderno

Desarrollo Web en Entorno Cliente · UD1

Objetivo. Practicar de forma progresiva los contenidos trabajados hasta la sesión 6: variables, operadores, condicionales, bucles, funciones, arrays, objetos, arrow functions, métodos modernos de arrays, reduce, destructuring y spread/rest.

Indicaciones: realiza los ejercicios en orden. A medida que avances, se indicará menos qué herramienta debes usar. Intenta reutilizar lo aprendido en los ejercicios anteriores y evita modificar arrays u objetos originales cuando se solicite una transformación.

BLOQUE 1 · FUNDAMENTOS

1. Temperaturas •

Dado el siguiente array:

```
const temperaturas = [18, 21, 24, 19, 27, 30, 22];
```

Realiza un programa que: a) muestre todas las temperaturas; b) calcule la máxima y la mínima; c) cuente cuántas son superiores a 20 ºC; d) calcule la media.

En este ejercicio no utilices map, filter ni reduce.

2. Funciones •

Crea las funciones esPar(numero), mayor(a, b) y calcularIVA(precio, porcentaje). Después transforma las tres en arrow functions.

```
calcularIVA(100, 21); // 21
```

3. Analizando un array ••

Dado el siguiente array:

```
const notas = [4, 7, 8, 3, 9, 5, 6, 10, 2];
```

Obtén: a) las notas aprobadas; b) las suspensas; c) la primera nota mayor o igual que 9; d) si existe algún 10; e) si todas las notas son válidas (entre 0 y 10).

Utiliza cuando corresponda filter, find, some y every.

BLOQUE 2 · ARRAYS Y OBJETOS

4. Productos ••

Dado el siguiente array de objetos:

```
const productos = [
  { id: 1, nombre: "Teclado", precio: 35, stock: 4 },
  { id: 2, nombre: "Ratón", precio: 18, stock: 0 },
  { id: 3, nombre: "Monitor", precio: 190, stock: 3 },
  { id: 4, nombre: "USB", precio: 12, stock: 10 },
```

```
  { id: 5, nombre: "Webcam", precio: 65, stock: 0 }
];
```

Obtén: a) productos con stock; b) producto con id 3; c) productos de menos de 50 €; d) un array únicamente con los nombres; e) un nuevo array con objetos que contengan nombre y precioConIVA.

El array original no debe modificarse.

5. Transformación de datos ••

Partiendo del array productos del ejercicio anterior, genera mediante map un array con cadenas del tipo:

```
["Teclado - 35 €", "Ratón - 18 €", ...]
```

Después genera únicamente las cadenas correspondientes a productos con stock. Intenta resolverlo encadenando métodos.

BLOQUE 3 · REDUCE

6. Total del carrito ••

Dado:

```
const carrito = [
  { producto: "Teclado", precio: 35, cantidad: 2 },
  { producto: "Ratón", precio: 18, cantidad: 1 },
  { producto: "Monitor", precio: 190, cantidad: 2 }
];
```

Utiliza reduce para calcular el importe total del carrito teniendo en cuenta precio y cantidad. Resultado esperado: 468.

7. Estadísticas en un único recorrido •••

Dado:

```
const numeros = [4, 7, 2, 9, 5, 8, 1, 6];
```

Utilizando reduce, obtén un objeto con las propiedades suma, pares e impares. No realices tres recorridos diferentes del array.

BLOQUE 4 · DESTRUCTURING Y SPREAD

8. Datos de usuario ••

Dado:

```
const usuario = {
  id: 17,
  nombre: "Lucía",
  email: "lucia@email.com",
  edad: 22,
  activo: true
};
```

Mediante destructuring: a) extrae nombre y email; b) extrae edad en una variable denominada edadUsuario; c) muestra: "Lucía (22 años) - lucia@email.com".

9. Actualización inmutable ••

Partiendo del objeto usuario anterior, crea un nuevo objeto usuarioActualizado que mantenga todas las propiedades, cambie edad a 23 y añada rol: "admin".

No debes modificar usuario. Después muestra ambos objetos para comprobarlo.

10. Arrays y spread ••

Dados:

```
const grupoA = ["Ana", "Luis", "Marta"];
const grupoB = ["Carlos", "Lucía"];
```

Crea mediante spread: a) un array con ambos grupos; b) otro que añada "Pedro" al principio; c) una copia de grupoA. Modifica posteriormente la copia y comprueba qué ocurre con el array original.

BLOQUE 5 · COMBINAMOS CONCEPTOS

11. Gestión de alumnos •••

Dado:

```
const alumnos = [
  { id: 1, nombre: "Ana", nota: 7.5 },
  { id: 2, nombre: "Luis", nota: 4 },
  { id: 3, nombre: "Marta", nota: 9 },
  { id: 4, nombre: "Carlos", nota: 5 },
  { id: 5, nombre: "Lucía", nota: 8.5 }
];
```

Resuelve: a) alumnos aprobados; b) nombres de aprobados; c) alumno con id 3; d) si existe algún suspenso; e) si todas las notas son válidas entre 0 y 10; f) nota media mediante reduce; g) un nuevo array en el que cada alumno tenga una propiedad aprobado con valor true o false.

No modifiques los objetos originales.

BLOQUE 6 · PROBLEMAS INTEGRADORES

12. Tienda •••

Utiliza de nuevo el array productos del ejercicio 4 e implementa las siguientes funciones:

```
buscarProducto(id)
productosDisponibles()
valorAlmacen()
aplicarDescuento(porcentaje)
```

Las funciones deberán permitir buscar por id, obtener productos con stock, calcular el valor total del almacén (precio × stock) y obtener un nuevo array con los precios rebajados.

aplicarDescuento() no puede modificar el array original.

13. Pedidos •••

Dado:

```
const pedidos = [
  { id: 1, cliente: "Ana", total: 120, pagado: true },
  { id: 2, cliente: "Luis", total: 75, pagado: false },
```

```
  { id: 3, cliente: "Marta", total: 210, pagado: true },
  { id: 4, cliente: "Carlos", total: 45, pagado: false },
  { id: 5, cliente: "Lucía", total: 160, pagado: true }
];
```

Sin utilizar bucles tradicionales, obtén: a) pedidos pagados; b) nombres de sus clientes; c) primer pedido superior a 150 €; d) importe total; e) importe pendiente de pago; f) si existe algún pedido superior a 200 €; g) si todos tienen importe positivo.

14. Reto final · Gestión de tareas ••••

Disponemos del siguiente array:

```
const tareas = [
  { id: 1, titulo: "Estudiar JavaScript", completada: true, prioridad: 2 },
  { id: 2, titulo: "Practicar arrays", completada: false, prioridad: 1 },
  { id: 3, titulo: "Repasar objetos", completada: false, prioridad: 3 }
];
```

Desarrolla las siguientes funciones. Ninguna función debe modificar el array recibido ni los objetos que contiene.

a) obtenerPendientes(tareas)

Recibe un array de tareas y devuelve un nuevo array que contenga únicamente las tareas que no están completadas.

```
obtenerPendientes(tareas);
```

Para los datos iniciales debe devolver las tareas con id 2 y 3.

b) buscarTarea(tareas, id)

Recibe un array de tareas y un identificador. Devuelve la tarea cuyo id coincida con el indicado.

```
buscarTarea(tareas, 2);
```

Debe devolver la tarea "Practicar arrays". Si no existe ninguna tarea con ese id, devolverá undefined.

c) completarTarea(tareas, id)

Recibe un array de tareas y el id de una tarea. Devuelve un nuevo array en el que la tarea indicada tenga completada: true. El resto de tareas debe permanecer sin cambios.

```
const nuevasTareas = completarTarea(tareas, 2);
```

En nuevasTareas, la tarea 2 estará completada, pero el array original tareas deberá seguir exactamente igual.

d) añadirTarea(tareas, nuevaTarea)

Recibe el array de tareas y una nueva tarea. Devuelve un nuevo array que contenga las tareas existentes y la nueva tarea al final.

```
const nueva = {
  id: 4,
  titulo: "Practicar reduce",
  completada: false,
  prioridad: 2
};

const nuevasTareas = añadirTarea(tareas, nueva);
```

El array nuevasTareas tendrá cuatro elementos y tareas seguirá teniendo tres.

e) calcularEstadisticas(tareas)

Recibe un array de tareas y devuelve un objeto con el número total de tareas, el número de completadas y el número de pendientes.

```
calcularEstadisticas(tareas);
```

Para el array inicial deberá obtenerse: { total: 3, completadas: 1, pendientes: 2 }.

Condiciones del reto

Intenta utilizar los métodos de arrays más adecuados (map, filter, find, reduce, etc.), junto con destructuring y spread cuando sean útiles.

No se permite modificar directamente el array ni los objetos originales.